DSA STUDY BLUEPRINT SERIES

51. Reverse a Linked List

Iterative and Recursive Linked List Reversals

Section 07 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Reversal:** Changing node pointer directions to point to predecessors.
- Use three tracking pointers: `prev`, `curr`, and `next`.

Memory & Execution Blueprint

C++ Implementation Reference

Standard code layout and syntax

```
Node* reverseList(Node* head) {
    Node *prev = nullptr, *curr = head, *nxt = nullptr;
    while (curr) {
        nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    }
    return prev;
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	O(N)
Space Complexity	O(1) iterative / O(N) recursive stack

Key Practices & Gotchas

- **Important:** Always check for empty lists or single node cases before processing.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace the re-wirings of three node pointers reversing segment `10 -> 20`:

Step	Pointers [prev, curr, next]	Pointer reassignment	Resulting node state
1	[nullptr, 10, 20]	`curr->next = prev`	10 points to nullptr
2	[10, 20, nullptr]	`curr->next = prev`	20 points to 10 (Reversed segment completed!)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pointer Re-linking:** Modifying node transitions requires dummy nodes to isolate head pointer alterations.
- **Fast & Slow Pointers:** Useful for loop check detections, middle-node finding, and cyclic lists splits.
- **Related LeetCode Problems:** #206 Reverse Linked List, #141 Linked List Cycle, #23 Merge k Sorted Lists.